

Cifras Modernas

Q1 — Impacto de NONCE Fixo no ChaCha20

Se for utilizado um NONCE fixo (por exemplo, tudo zeros) com a mesma chave, o keystream gerado pelo ChaCha20 será sempre o mesmo para todas as mensagens cifradas. Isto tem implicações graves para a segurança:

Sendo o ChaCha20 baseado em XOR:

$C1 = M1 \text{ XOR Keystream}$

$C2 = M2 \text{ XOR Keystream}$

Um atacante que intercepte dois criptogramas pode calcular:

$C1 \text{ XOR } C2 = M1 \text{ XOR } M2$

O keystream cancela-se, expondo a relação entre as duas mensagens originais. Este ataque é conhecido como Two-Time Pad e foi estudado na semana anterior com o OTP. Conhecendo padrões de linguagem natural (palavras comuns, espaços, estrutura), é possível recuperar ambas as mensagens.

Em resumo, um NONCE fixo destrói completamente a segurança da cifra ChaCha20, tornando-a vulnerável ao mesmo ataque que afeta o OTP quando a chave é reutilizada. O NONCE deve ser sempre gerado aleatoriamente e usado uma única vez por mensagem com a mesma chave.

Q2 — Difusão no ChaCha20

A cifra ChaCha20, por se tratar de uma cifra sequencial síncrona baseada em XOR, não promove difusão. Para verificar este comportamento, foram criadas duas mensagens que diferem apenas num byte e cifradas com a mesma chave e mesmo NONCE:

`msg1 = b"AAAAAAAAAAAAAAAAAAAA"`

`msg2 = b"BAAAAAAAAAAAAAAAAAAAAA" # apenas o primeiro byte é diferente`

Resultado obtido:

Medição	Valor
Bits diferentes no texto-limpo	2 bits (A vs B)
Bytes diferentes no criptograma	1 byte
Bits diferentes no criptograma	2 bits

Como o ChaCha20 usa XOR com um keystream fixo, a diferença no criptograma é exatamente igual à diferença no texto-limpo — alterar 1 bit no texto-limpo afeta exatamente

1 bit no criptograma (neste caso 2 bits porque A e B diferem em 2 bits). Não existe efeito avalanche.

Esta ausência de difusão significa que o ChaCha20 não garante integridade dos dados — um atacante que conheça o conteúdo de uma posição do texto-limpo pode manipular o criptograma para alterar esse conteúdo sem conhecer a chave, como demonstrado pelo programa `chacha20_int_attck.py`.

Q3 — Bit Flips nos Modos CBC e CTR

Foi realizado um teste alterando 1 bit num bloco do criptograma e analisando o impacto no texto decifrado para cada modo:

Resultado obtido:

Modo	Bits afetados	Motivo
CTR	1 bit	XOR puro — mudanças propagam 1:1, sem difusão
CBC	~62 bits (~metade do bloco + 1 bit)	AES mistura o bloco inteiro + propaga 1 bit para o bloco seguinte

Modo CTR

O modo CTR comporta-se como uma cifra sequencial — gera um keystream via AES e faz XOR com o texto-limpo. Alterar 1 bit no criptograma afeta exatamente 1 bit no texto decifrado, sem qualquer difusão, tal como o ChaCha20.

Modo CBC

O modo CBC tem encadeamento entre blocos. Alterar 1 bit num bloco do criptograma tem dois efeitos ao decifrar:

- O bloco correspondente fica completamente corrompido (efeito avalanche do AES — cerca de metade dos bits mudam)
- O bloco seguinte sofre exatamente 1 bit de corrupção (propagação do XOR de encadeamento)
- Os restantes blocos ficam completamente intactos

Esta propriedade do CBC oferece uma proteção acidental contra ataques de manipulação — a corrupção do bloco tende a invalidar o padding, causando erros na decifração. No entanto, nenhum dos modos garante integridade real — para isso é necessário autenticação (AEAD).

Q4 — Impacto do `chacha20_int_attck.py` no AES-CBC e AES-CTR

O programa `chacha20_int_attck.py` realiza um ataque de manipulação de criptograma usando XOR direto. O seu impacto difere consoante o modo de operação:

AES-CTR

O ataque funciona de forma idêntica ao ChaCha20, uma vez que o modo CTR é essencialmente uma cifra sequencial baseada em XOR. A manipulação de um byte no criptograma afeta exatamente o byte correspondente no texto decifrado, permitindo ao atacante substituir conteúdo conhecido por conteúdo arbitrário sem conhecer a chave.

Resultado experimental:

```
Texto original:  TRANSFERE 1000 EUROS PARA CONTA
Após ataque:    TRANSFERE 1000 E][FZ PARA CONTA
```

O ataque alterou com precisão os bytes na posição indicada, confirmando a ausência de difusão no modo CTR.

AES-CBC

O ataque falha com um erro de padding inválido. Isto acontece porque a manipulação de bytes no criptograma CBC corromperia o bloco inteiro ao decifrar (efeito avalanche do AES), propagando a corrupção para o padding do bloco seguinte. O validador de padding deteta a inconsistência e rejeita a decifração.

Embora o CBC ofereça esta proteção acidental contra o ataque, não é uma garantia de integridade — é apenas um efeito colateral do encadeamento e do padding. Para garantir integridade real é necessário usar cifras autenticadas (AEAD) como ChaCha20-Poly1305 ou AES-GCM.

Modo	Resultado do ataque	Motivo
CTR	Funciona — bytes alterados com precisao	XOR puro, sem difusao
CBC	Falha — erro de padding invalido	Corrupcao do bloco invalida o padding

Q5 — Função do Salt e do NONCE no PBE ChaCha20

Na versão de Password-Based Encryption do ChaCha20, o ficheiro cifrado contém três componentes:

```
ficheiro.enc = [ Salt (16 bytes) ] + [ NONCE (16 bytes) ] + [ Criptograma ]
```

Função do Salt

O salt é um valor aleatório gerado na cifragem e armazenado junto com o criptograma. A sua função é garantir que a mesma password produza chaves diferentes em cada cifragem, através da KDF PBKDF2:

```
PBKDF2("ssi2526", Salt_1) -> Chave_A
PBKDF2("ssi2526", Salt_2) -> Chave_B (completamente diferente)
```

Sem salt, um atacante poderia pré-calcular chaves para passwords comuns (ataque de dicionário), comprometendo todos os ficheiros cifrados com essas passwords. O salt torna este ataque inviável computacionalmente.

Função do NONCE

O NONCE é usado pelo ChaCha20 para garantir que o keystream é único mesmo que a chave derivada seja igual. Sem NONCE, cifrar dois ficheiros com a mesma password e mesmo salt produziria o mesmo keystream, tornando o sistema vulnerável ao ataque Two-Time Pad.

São ambos necessários?

Sim, ambos são necessários — protegem camadas diferentes e independentes do sistema:

	Salt	NONCE
Usado em	KDF (derivacao da chave)	ChaCha20 (cifragem)
Protege contra	Ataques de dicionario pre-computado	Reutilizacao do keystream (Two-Time Pad)
Secreto?	Nao — guardado com o criptograma	Nao — guardado com o criptograma
Camada	Derivacao da chave	Cifra

O salt protege a derivacao da chave e o NONCE protege a cifra. Remover qualquer um deles introduz vulnerabilidades distintas — sao complementares e ambos essenciais para a seguranca do sistema.